

1. Write program demonstrates how to use regular expressions in Python to match and search for patterns in text.

```
import re

text = input("Enter the text: ")
pattern = input("Enter the pattern: ")

result = re.search(pattern, text)

if result:
    print("Pattern Found:", result.group())
else:
    print("Pattern Not Found")
```

```
Enter the text: mobile number is 9390727569
Enter the pattern: \d{10}
Pattern Found: 9390727569
```

2. Implement a basic finite state automaton that recognizes a specific language or pattern. In this example, we'll create a simple automaton to match strings ending with 'ab' using python.

```
def fsa(string):
    if len(string) >= 2 and string[-2:] == "ab":
        return True
    return False

s = input("Enter a string: ")

if fsa(s):
    print("Accepted")
else:
    print("Rejected")
```

```
Enter a string: aaaab
Accepted
```

3. Write program demonstrates how to perform morphological analysis using the NLTK library in Python.

```
import nltk
from nltk.stem import WordNetLemmatizer

nltk.download('wordnet')

lemmatizer = WordNetLemmatizer()

sentence = input("Enter a sentence: ")

words = sentence.split()

print("\nMorphological Analysis:")
for word in words:
    print(word, "->", lemmatizer.lemmatize(word))
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
Enter a sentence: welcome to saveetha university

Morphological Analysis:
welcome -> welcome
to -> to
saveetha -> saveetha
university -> university
```

4. Implement a finite-state machine for morphological parsing. In this example, we'll create a simple machine to generate plural forms of English nouns using python.

```
word = input("Enter a singular noun: ")

if word.endswith("y"):
    plural = word[:-1] + "ies"
elif word.endswith(("s", "x", "z", "ch", "sh")):
    plural = word + "es"
```

```

else:
    plural = word + "s"

print("Plural Form:", plural)

```

Enter a singular noun: city
Plural Form: cities

5. Use the Porter Stemmer algorithm to perform word stemming on a list of words using python libraries.

```

import nltk
from nltk.stem import PorterStemmer

ps = PorterStemmer()

text = input("Enter words separated by spaces: ")

words = text.split()

print("\nStemmed Words:")
for word in words:
    print(word, "->", ps.stem(word))

```

Enter words separated by spaces: playing eating walking watching

Stemmed Words:
playing -> play
eating -> eat
walking -> walk
watching -> watch

6. Implement a basic N-gram model for text generation. For example, generate text using a bigram model using python.

```

import random

text = input("Enter a sentence: ")

words = text.split()

bigrams = {}

for i in range(len(words)-1):
    if words[i] not in bigrams:
        bigrams[words[i]] = []
    bigrams[words[i]].append(words[i+1])

start = random.choice(words[:-1])
result = [start]

for i in range(5):
    if start in bigrams:
        start = random.choice(bigrams[start])
        result.append(start)
    else:
        break

print("Generated Text:")
print(" ".join(result))

```

Enter a sentence: SIMATS ENGINEERING welcomes you
Generated Text:
ENGINEERING welcomes you

7. Write program using the NLTK library to perform part-of-speech tagging on a text.

```

import nltk

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')

text = input("Enter a sentence: ")

```

```

words = nltk.word_tokenize(text)

tags = nltk.pos_tag(words)

print("\nPOS Tags:")
for word, tag in tags:
    print(word, ":", tag)

[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] /root/nltk_data...
[nltk_data] Unzipping taggers/averaged_perceptron_tagger.zip.
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /root/nltk_data...
[nltk_data] Unzipping taggers/averaged_perceptron_tagger_eng.zip.
Enter a sentence: i love watching movies

POS Tags:
i : NN
love : VBP
watching : VBG
movies : NNS

```

8. Implement a simple stochastic part-of-speech tagging algorithm using a basic probabilistic model to assign POS tags using python

```

import random

noun = ["dog", "cat", "book", "student"]
verb = ["run", "eat", "play", "write"]
adj = ["good", "big", "small", "beautiful"]

sentence = input("Enter words separated by spaces: ")

words = sentence.split()

print("\nPOS Tags:")

for word in words:
    if word in noun:
        print(word, ": NOUN")
    elif word in verb:
        print(word, ": VERB")
    elif word in adj:
        print(word, ": ADJECTIVE")
    else:
        print(word, ":", random.choice(["NOUN", "VERB", "ADJECTIVE"]))

Enter words separated by spaces: dog play apple beautiful

POS Tags:
dog : NOUN
play : VERB
apple : NOUN
beautiful : ADJECTIVE

```

9. Implement a rule-based part-of-speech tagging system using regular expressions using python.

```

import re

sentence = input("Enter a sentence: ")

words = sentence.split()

print("\nPOS Tags:")

for word in words:
    if re.fullmatch(r".*ing", word):
        print(word, ": Verb")
    elif re.fullmatch(r".*ed", word):
        print(word, ": Verb")
    elif re.fullmatch(r".*ly", word):
        print(word, ": Adverb")
    elif re.fullmatch(r".*ness", word):

```

```
print(word, ": Noun")
else:
    print(word, ": Noun")
```

Enter a sentence: i love reading books

POS Tags:
i : Noun
love : Noun
reading : Verb
books : Noun

10. Implement transformation-based tagging using a set of transformation rules, apply a simple rule to tag words using python

```
sentence = input("Enter a sentence: ")
words = sentence.split()

print("\nTransformation-Based Tags:")

for word in words:
    tag = "NOUN"

    if word.endswith("ing"):
        tag = "VERB"
    elif word.endswith("ly"):
        tag = "ADVERB"
    elif word.endswith("ed"):
        tag = "VERB"
    elif word[0].isupper():
        tag = "PROPER NOUN"

    print(word, ":", tag)
```

Enter a sentence: we are studying BTECH AI&DS 3rd Year

Transformation-Based Tags:
we : NOUN
are : NOUN
studying : VERB
BTECH : PROPER NOUN
AI&DS : PROPER NOUN
3rd : NOUN
Year : PROPER NOUN